

Izračunavanje i vizuelizacija poligona vidljivosti

Tijana Jakšić

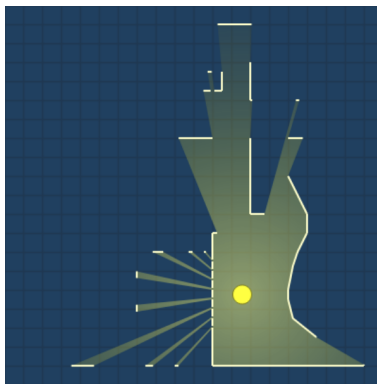
14. septembar 2024.

Sadržaj

1	Uvod	1
2	Definicija problema	1
3	Implementacija	2
3.1	Strukture podataka	2
3.1.1	Kontejnerske strukture podataka	3
3.2	Algoritam	4
4	Analiza složenosti	5
4.1	Vremenska složenost	5
5	Prostorna složenost	5

1 Uvod

Ukoliko je data mapa ravni posmatrane odozgo sa nekim preprekama u vidu segmenata, i tačka posmatranja p , poligon vidljivosti predstavlja podskup te ravni koji opisuje sve tačke vidljive iz p .



Slika 1: Ilustracija vidljivog regiona iz date tačke [2]

Poligon vidljivosti ima veze sa poznatim problemom umetničke galerije, a nalazi praktičnu primenu u video igrama, robotici i navigaciji.

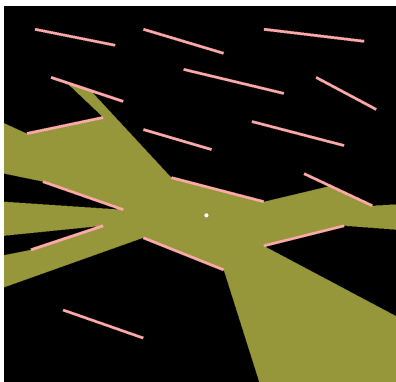
Prvi rad koji opisuje pronalaženje poligona vidljivosti primenom tehnike rotacionog brišućeg zraka je [1]. Ovaj projekat implementira sličan algoritam u programskom jeziku C++, oslanjajući se na biblioteku SFML radi vizuelizacije. Implementirana je interaktivna vizuelizacija u kojoj korisnik položajem miša zadaje koordinate tački posmatranja. Osim toga, moguće je zadati koordinate preprekama, menjanjem fajla `segments.txt` pre pokretanja programa.

2 Definicija problema

Matematički, poligon vidljivosti se može uvesti na sledeći način: Neka su u ravni $\mathbf{R}^2$ dati skup duži S (*prepreke*) i tačka p koja se ne nalazi na nekoj od prepreka. Poligon vidljivosti V je tada skup tačaka q u $\mathbf{R}^2$ takvih da segment pq nema preseka ni sa jednom preprekom.

Primetimo da ovako definisan, poligon vidljivosti može biti neograničen podskup ravni. Prva dodatna pretpostavka koja se pravi radi implementacije jeste da je skup S konačan. Dalje pretpostavljamo da ne radimo sa celom ravni, već da su u skupu prepreka dati segmenti koji prave zatvorenu regiju u kojoj se svi ostali segmenti i tačka posmatranja nalaze. Ovo je u redu jer je bilo kom konačnom skupu segmenata moguće naknadno dodati granicu.

Osim toga, pretpostavlja se da se datim segmentima ne seku unutrašnjosti. U slučaju datih duži koje imaju preseke u unutrašnjim tačkama, bilo bi potrebno preprocesirati ulazni skup duži, ali u ovom projektu to nije implementirano.



Slika 2: Poligon vidljivosti

Dakle, algoritam za ulaz prima skup prepreka i tačku posmatranja koji zadovoljavaju sledeće pretpostavke:

- Prepreke su segmenti kojima se unutrašnjosti ne presecaju
- Postoji podskup segmenata koji čini zatvorenu regiju, u kojoj se nalaze svi ostali segmenti i tačka posmatranja

Kao izlaz, algoritam daje vektor tačaka koji opisuje poligon vidljivosti.

3 Implementacija

3.1 Strukture podataka

Pomoćne geometrijske strukture podataka:

Struktura: Point

x: float
y: float

Struktura: Segment

a: Point
b: Point

Struktura podataka koja opisuje događaj:

Struktura: Event

point: Point
segment: Segment
end: bool

Atribut *end* čuva informaciju o tome da li je *point* početno ili krajnje teme segmenta. Ovo se može utvrditi na osnovu orijentacije trougla $(p, \text{segment}.a, \text{segment}.b)$.

3.1.1 Kontejnerske strukture podataka

Red događaja. Događaji su smeštani u vektor $\mathcal{E}$. Kako je za tehniku brišućeg zraka potrebno sortirati događaje, uveden je komparator događaja `CompareEvents`. Glavni kriterijum je ugao vektora $p \rightarrow e.\text{point}$. Ukoliko ima više događaja na istom zraku posmatranja, ova implementacija tretira udaljenije događaje kao manje, potom ako su identični i ugao i udaljenost, tretira *end* događaje kao manje.

Status. Status je struktura podataka $\mathcal{T}$ koja treba da čuva niz duži koje seku rotacioni brišućí zrak, ispravno poredan tako da $\min \mathcal{T}$ predstavlja prvi vidljiv segment. Osim toga, $\mathcal{T}$ treba da omogućava efikasnu pretragu, umetanje i brisanje elemenata. Zato se koristi uređeno balansirano binarno stablo. Uveden je komparator `CompareSegments` koji opisuje relaciju „ispred“. Relacija zavisi od tačke posmatranja p . Primetimo da relacija ispred nije isto što i relacija udaljenosti od p .



Slika 3: Dva segmenta kojima se relacija ispred ne poklapa sa udaljenošću od p

Za segment s_1 važi da je ispred segmenta s_2 ukoliko postoji zrak posmatranja r iz račke posmatranja p koji seče oba segmenta i, ukoliko su ti preseči I_1 i I_2 , važi $d(I_1, p) < d(I_2, p)$. Drugim rečima, ukoliko je udaljenost do tačke posmatranja duž zrak posmatranja manja. Upravo zbog ovoga je važno da se datim segmentima ne seku unutrašnjosti - da bi ova relacija bila doro definisana.

3.2 Algoritam

U nastavku je opisan glavni algoritam, koji kao ulaz prima tačku posmatranja p i skup segmenata prepreka $\mathcal{S}$, a kao izlaz vraća vektor čvorova poligona vidljivosti $\mathcal{V}$.

Algoritam: VisibilityPolygon($p, \mathcal{S}$)

- 1: Inicijalizuj prazan vektor događaja $\mathcal{E}$
- 2: **za svaki** segment s u $\mathcal{S}$ **radi**
- 3: Za svaki čvor segmenta s dodaj odgovarajući događaj u $\mathcal{E}$
- 4: Sortiraj $\mathcal{E}$ po kriterijumu CompareEvents
- 5: Inicijalizuj praznu strukturu za status $\mathcal{T}$
- 6: Inicijalizuj prazan vektor $\mathcal{V}$
- 7: **za svaki** e u $\mathcal{E}$ **radi**
- 8: ProcessEvent(e)

Dalje je opisan algoritam obrade događaja, koji za ulaz prima događaj e , i na osnovu toga inkrementalno ažurira $\mathcal{V}$. Ugao posmatranja je ugao koji formira vektor $p \rightarrow e.point$, a zrak posmatranja poluprava koja polazi iz p i ima ugao posmatranja.

Algoritam: ProcessEvent(e)

- 1: Ažuriraj ugao posmatranja
- 2: **ako je** $e.end = \text{true}$ **onda**
- 3: Obriši $e.segment$ iz $\mathcal{T}$
- 4: **ako je** $e.segment = \min(\mathcal{T})$ **onda** {segment je bio vidljiv}
- 5: Dodaj $e.point$ u $\mathcal{V}$
- 6: **ako je** $\mathcal{T} \neq \emptyset$ **onda**
- 7: Dodaj u $\mathcal{V}$ presek novog $\min(\mathcal{T})$ sa zrakom posmatranja
- 8: **inače** { $e.end = \text{false}$ }
- 9: **ako je** $\mathcal{T} \neq \emptyset$ **onda**
- 10: Pronađi tačku q preseka $\min(\mathcal{T})$ sa zrakom posmatranja
- 11: Dodaj $e.segment$ u $\mathcal{T}$
- 12: **ako je** $e.segment = \min(\mathcal{T})$ **onda** {novi najbliži segment}
- 13: **ako je** tačka q je pronađena **onda**
- 14: Dodaj q u $\mathcal{V}$
- 15: Dodaj $e.point$ u $\mathcal{V}$

4 Analiza složenosti

4.1 Vremenska složenost

Neka je broj datih segmenata prepreka n . Glavni algoritam `VisibilityPolygon` učitava događaje u $O(n)$ vremena, sortira ih u $O(n \log n)$ vremena, i potom $O(n)$ puta primeni `ProcessEvent`. Jedine operacije u algoritmu `ProcessEvent` koje se ne izvršavaju u linearnom vremenu su ažuriranje i pretraga statusa $\mathcal{T}$, koji se izvršavaju u logaritamskom vremenu. Kako se u $\mathcal{T}$ može nalaziti najviše n elemenata, znači da se algoritam `VisibilityPolygon` izvršava u $O(n \log n)$ vremena.

5 Prostorna složenost

Dodatne strukture podataka koje se koriste su red događaja u kome se može nalaziti najviše $2n$ elemenata (po jedan događaj za svaki čvor segmenta), i balansirano uređeno binarno stablo $\mathcal{T}$ koje čuva segmente, te može imati najviše n elemenata.

Ova implementacija rezultat pre crtanja čuva u vektoru, ali to nije neophodno, moguće je crtati trougaone podskupove rezultata jedan po jedan. Ova odluka ne utiče na asimptotsku prostornu složenost, zato što rezultat ne može imati više od $4n$ elemenata (događaja je $2n$ i svaki događaj dodaje najviše 2 tačke).

Dakle, prostorna složenost ovog algoritma je linearna.

Literatura

- [1] Tetsuo Asano. An efficient algorithm for finding the visibility polygon for a polygonal region with holes. *Institute of Electronics, Information, and Communication Engineers*, 68:557–559, 1985.
- [2] Red Blob Games. 2d visibility. <https://www.redblobgames.com/articles/visibility/>, 2012. Accessed: 14.9.2026.